

Practical No: 8

1. Title

Integration of JSP with Servlet using MVC Architecture

2. Aim

To develop a dynamic web application by integrating JSP and Servlet using MVC architecture, where the Servlet acts as a controller, JSP is used as the view, and Java classes are used as the model.

3. Learning Objectives

- To understand MVC architecture in web applications
- To learn integration of JSP and Servlet
- To understand Servlet as Controller
- To understand JSP as View
- To implement Model classes (JavaBeans/POJO)
- To understand data flow between Model, View, and Controller
- To develop a simple MVC-based web application

4. Software Requirements

- Web Server: Apache Tomcat
- JDK: Version 8 or above
- Browser: Chrome / Firefox
- Database (Optional): MySQL / Oracle

5. Theory Background

MVC (Model-View-Controller) is a design pattern that separates application into three parts:

Components

- **Model** → Handles data and business logic (Java classes / database)
- **View** → JSP pages for displaying output
- **Controller** → Servlet that processes request and controls flow

Advantages

- Separation of concerns
- Easy maintenance
- Code reusability
- Better application control

Limitations

- Complex for beginners

- Requires understanding of flow
- More files involved

6. Assignment Questions

8.1. A college wants to develop a web-based system where students can enter their marks and view results. Design an MVC-based application where:

- JSP form accepts student name and marks (3 subjects).
- Servlet calculates total and percentage.
- Model class stores student data.
- JSP displays result.

Input:

Student.java

```
public class Student {
    private String name;
    private int m1, m2, m3;
    private int total;
    private double percentage;

    public Student(String name, int m1, int m2, int m3) {
        this.name = name;
        this.m1 = m1;
        this.m2 = m2;
        this.m3 = m3;

        this.total = m1 + m2 + m3;
        this.percentage = total / 3.0;
    }

    public String getName() {
        return name;
    }

    public int getTotal() {
        return total;
    }

    public double getPercentage() {
        return percentage;
    }
}
```

Index.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
```

```
<!DOCTYPE html>
<html>
<body>

<h2>Student Marks Entry</h2>

<form action="StudentServlet" method="post">
  Name: <input type="text" name="name"><br><br>

  Subject 1: <input type="number" name="m1"><br><br>
  Subject 2: <input type="number" name="m2"><br><br>
  Subject 3: <input type="number" name="m3"><br><br>

  <input type="submit" value="Calculate">
</form>

</body>
</html>
```

StudentServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class StudentServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String name = request.getParameter("name");
        int m1 = Integer.parseInt(request.getParameter("m1"));
        int m2 = Integer.parseInt(request.getParameter("m2"));
        int m3 = Integer.parseInt(request.getParameter("m3"));
        Student s = new Student(name, m1, m2, m3);
        request.setAttribute("student", s);

        RequestDispatcher rd = request.getRequestDispatcher("result.jsp");
        rd.forward(request, response);
    }
}
```

Result.jsp

```
<%@ page import="Student" %>

<html>
<body>
```

```
<h2>Student Result</h2>
```

```
<%
```

```
    Student s = (Student) request.getAttribute("student");
```

```
%>
```

```
Name: <%= s.getName() %> <br><br>
```

```
Total: <%= s.getTotal() %> <br><br>
```

```
Percentage: <%= s.getPercentage() %> %
```

```
</body>
```

```
</html>
```

Output:

Student Marks Entry

Student Result

Name:

Name: Shriyash Patil

Subject 1:

Total: 267

Subject 2:

Percentage: 89.0 %

Subject 3:

8.2. A company wants a login system for employees.

1. Create a login page using JSP. 2. Servlet should validate: ○ Username: admin ○ Password: admin123 3. If valid → redirect to dashboard JSP 4. If invalid → show error message on same page

Input:

Index.jsp

```
<html>
```

```
<body>
```

```
<h2>Employee Login</h2>
```

```
<form action="LoginServlet" method="post">
```

```
Username: <input type="text" name="username"><br><br>
```

```
Password: <input type="password" name="password"><br><br>
```

```
<input type="submit" value="Login">
```

```
</form>
```

```
<%
```

```
String error = (String) request.getAttribute("error");
```

```
if(error != null){
```

Python Programming Lab – MCA Sem II

```
%>
<p style="color:red;"><%= error %></p>
<% } %>
</body>
</html>
```

LoginServlet.java

```
import java.io.IOException;
import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
@WebServlet("/LoginServlet")
public class LoginServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
        String username = request.getParameter("username");
        String password = request.getParameter("password");
        if (username.equals("admin") && password.equals("admin123")) {
            response.sendRedirect("dashboard.jsp");
        } else {
            request.setAttribute("error", "Invalid Username or Password");
            RequestDispatcher rd = request.getRequestDispatcher("login.jsp");
            rd.forward(request, response);
        }
    }
}
```

Dashboard.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<body>
<h2>Welcome to Dashboard</h2>
<p>Login Successful!</p>
</body>
</html>
```

Output:

Welcome to Dashboard

Sh Login Successful!

Employee Login

Username:

Password:

8.3. A shop wants a billing system where users select products and quantity.

1. JSP form to input:

- Product name
- Price
- Quantity

2. Servlet calculates:

- Total amount
- GST (18%)
- Final bill

3. Model class stores product details

4. JSP displays bill in formatted table

Input:

Bill.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<body>

<h2>Shop Billing System</h2>

<form action="BillingServlet" method="post">
    Product Name: <input type="text" name="name" required><br><br>
    Price: <input type="number" name="price" step="0.01" required><br><br>
    Quantity: <input type="number" name="quantity" required><br><br>

    <input type="submit" value="Generate Bill">
</form>

</body>
</html>
```

Product.java

```
package com;
```

```
public class Product {  
  
    private String name;  
    private double price;  
    private int quantity;  
    private double total;  
    private double gst;  
    private double finalAmount;  
  
    public Product(String name, double price, int quantity) {  
        this.name = name;  
        this.price = price;  
        this.quantity = quantity;  
        this.total = price * quantity;  
        this.gst = total * 0.18;  
        this.finalAmount = total + gst;  
    }  
  
    public String getName() { return name; }  
    public double getPrice() { return price; }  
    public int getQuantity() { return quantity; }  
    public double getTotal() { return total; }  
    public double getGst() { return gst; }  
    public double getFinalAmount() { return finalAmount; }  
}
```

BillingServlet.java

```
package com;  
  
import java.io.IOException;  
  
import javax.servlet.RequestDispatcher;  
import javax.servlet.ServletException;  
import javax.servlet.annotation.WebServlet;  
import javax.servlet.http.HttpServlet;  
import javax.servlet.http.HttpServletRequest;  
import javax.servlet.http.HttpServletResponse;  
@WebServlet("/BillingServlet")  
public class BillingServlet extends HttpServlet {  
    private static final long serialVersionUID = 1L;  
  
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws  
ServletException, IOException {  
        String name = request.getParameter("name");
```

```
double price = Double.parseDouble(request.getParameter("price"));
int quantity = Integer.parseInt(request.getParameter("quantity"));
```

```
Product p = new Product(name, price, quantity);
```

```
request.setAttribute("product", p);
```

```
RequestDispatcher rd = request.getRequestDispatcher("result2.jsp");
rd.forward(request, response);
}
```

```
protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {
    doGet(request, response);
}

}
```

Result.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<%@ page import="com.Product" %>

<html>
<body>

<h2>Bill Receipt</h2>

<%
    Product p = (Product) request.getAttribute("product");
%>

<table border="1" cellpadding="10">
    <tr>
        <th>Product</th>
        <th>Price</th>
        <th>Quantity</th>
        <th>Total</th>
    </tr>

    <tr>
        <td><%= p.getName() %></td>
```



```
<td><%= p.getPrice() %></td>
<td><%= p.getQuantity() %></td>
<td><%= p.getTotal() %></td>
</tr>

<tr>
<td colspan="3">GST (18%)</td>
<td><%= p.getGst() %></td>
</tr>

<tr>
<td colspan="3"><b>Final Amount</b></td>
<td><b><%= p.getFinalAmount() %></b></td>
</tr>
</table>

</body>
</html>
```

Output:

Shop Billing System

Product Name:

Price:

Quantity:

Bill Receipt

Product	Price	Quantity	Total
PlayStation 5	45000.0	1	45000.0
GST (18%)			8100.0
Final Amount			53100.0

5. Conclusion

The experiments demonstrated the integration of JSP and Servlet using the MVC architecture to build dynamic web applications. By separating the Model, View, and Controller, the applications became more organized, maintainable, and scalable. Through practical implementations like student result system, login system, and billing system, it is clear that MVC improves code structure, reusability, and efficiency in Java web development.